Intro to SSRF

What is an SSRF?

SSRF stands for Server-Side Request Forgery. It's a vulnerability that allows a malicious user to cause the webserver to make an additional or edited HTTP request to the resource of the attacker's choosing.

Types of SSRF

There are two types of SSRF vulnerability; the first is a regular SSRF where data is returned to the attacker's screen. The second is a Blind SSRF vulnerability where an SSRF occurs, but no information is returned to the attacker's screen.

A successful SSRF attack can result in any of the following:

- Access to unauthorised areas.
- Access to customer/organisational data.
- Ability to Scale to internal networks.
- Reveal authentication tokens/credentials.

SSRF Examples

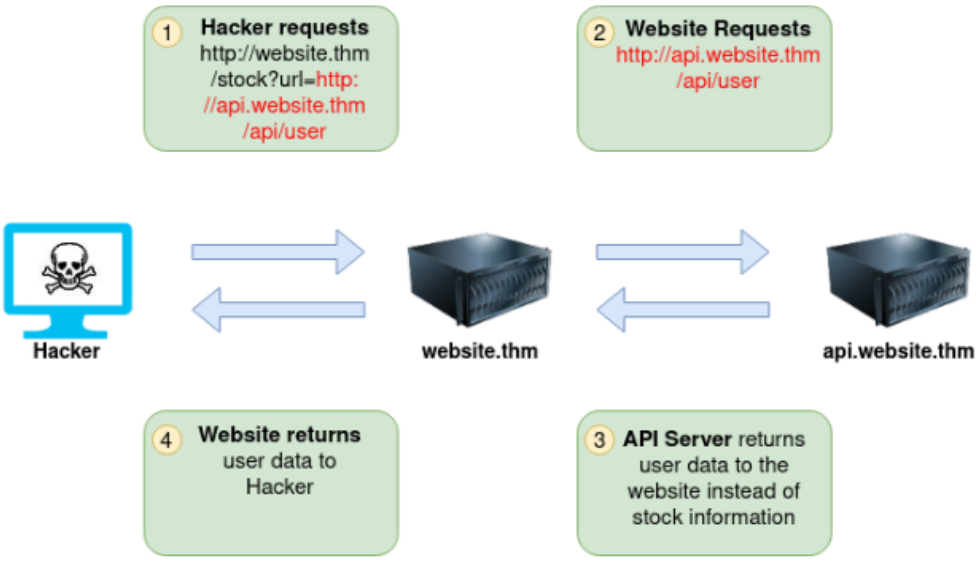
1#

Instructions

The below example shows how the attacker can have complete control over the page requested by the webserver.
The Expected Request is what the website.thm server is expecting to receive, with the section in red being the URL that the website will fetch for the information.
The attacker can modify the area in red to an URL of their choice. Next

Expected Request

http://website.thm/stock?url=http://api.website.thm/api/stock/item?id=123



2#

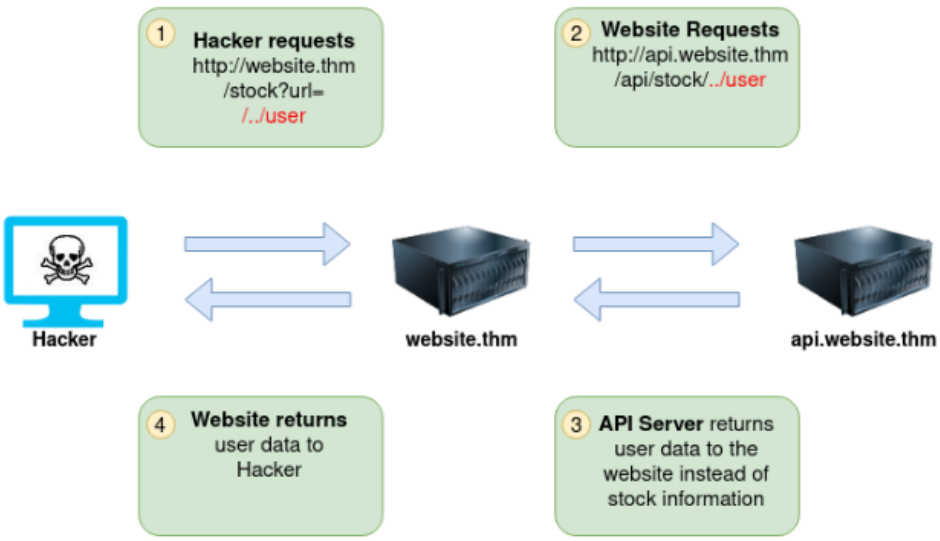
Instructions

The below example shows how an attacker can still reach the /api/user page with only having control over the path by utilising directory traversal. When website.thm receives ../ this is a message to move up a directory which removes the /stock portion of the request and turns the final request into /api/user

Next

Expected Request

http://website.thm/stock?url=/item?id=123



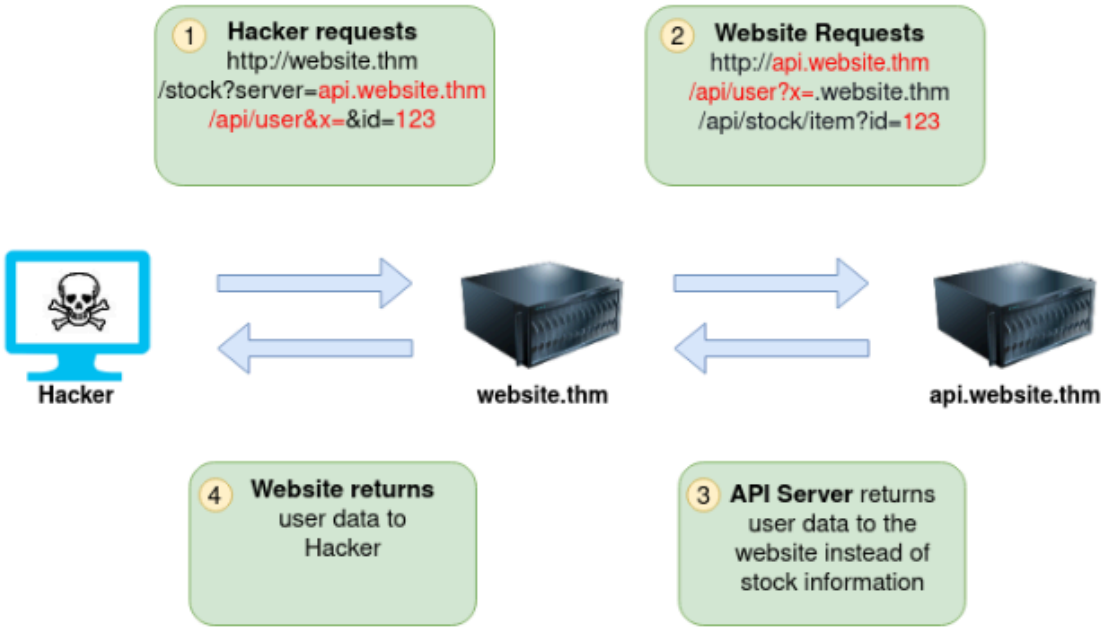
3#

Instructions

In this example, the attacker can control the server's subdomain to which the request is made. Take note of the payload ending in **&x=** being used to stop the remaining path from being appended to the end of the attacker's URL and instead turns it into a parameter (?x=) on the query string. Next

Expected Request

http://website.thm/stock?server=api&id=123

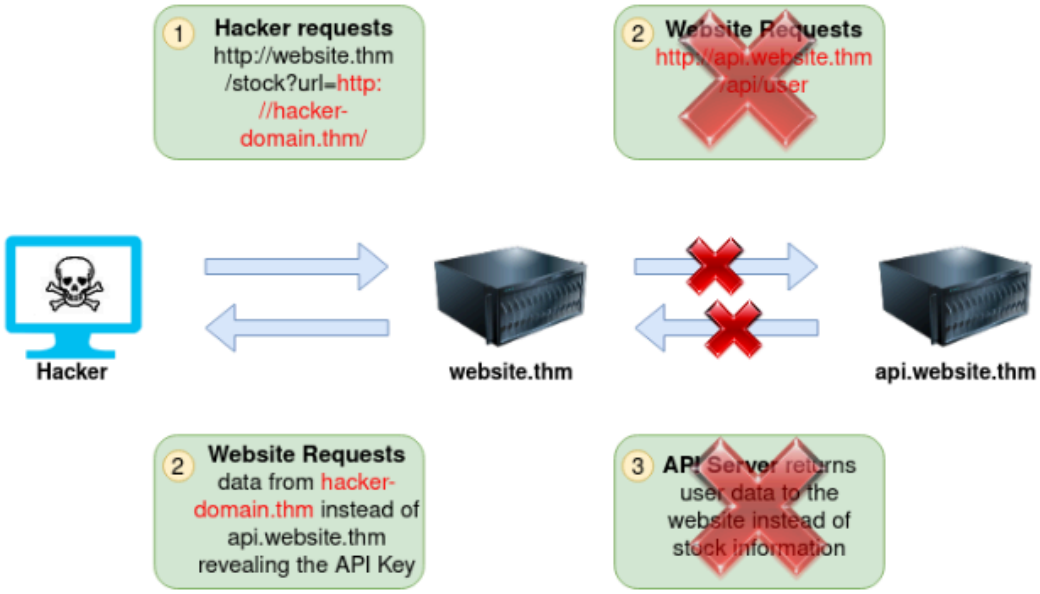


Instructions

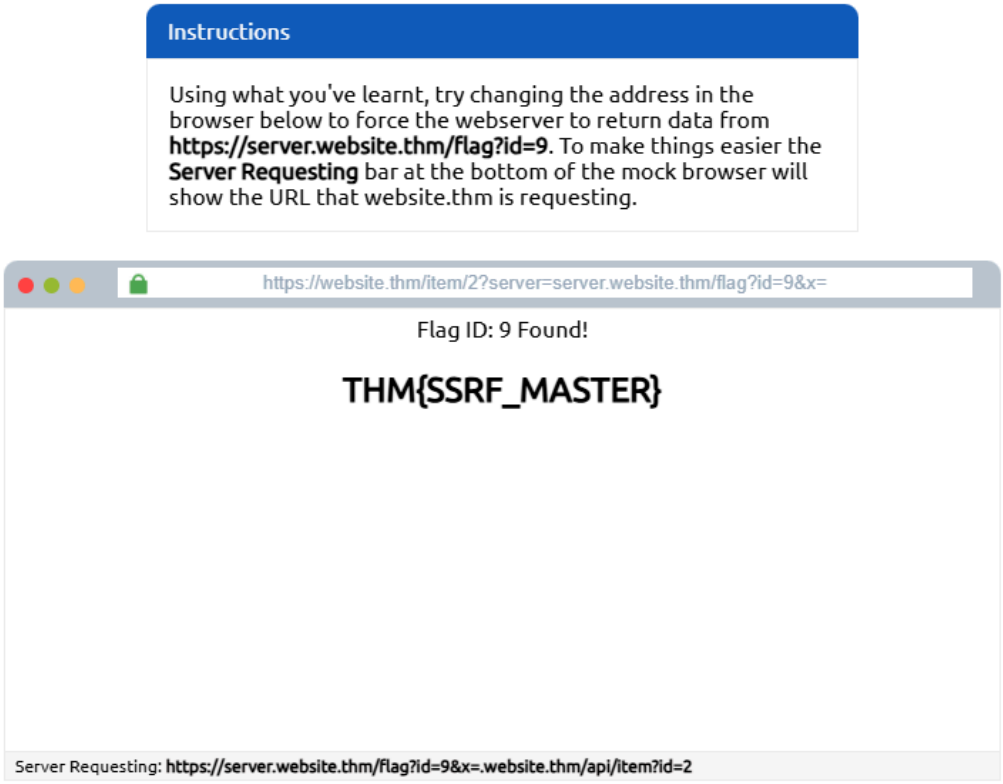
Going back to the original request, the attacker can instead force the webserver to request a server of the attacker's choice. By doing so, we can capture request headers that are sent to the attacker's specified domain. These headers could contain authentication credentials or API keys sent by website.thm (that would normally authenticate to api.website.thm). Next

Expected Request

http://website.thm/stock?url=http://api.website.thm/api/stock/item?id=123



Practical example I had to figure out

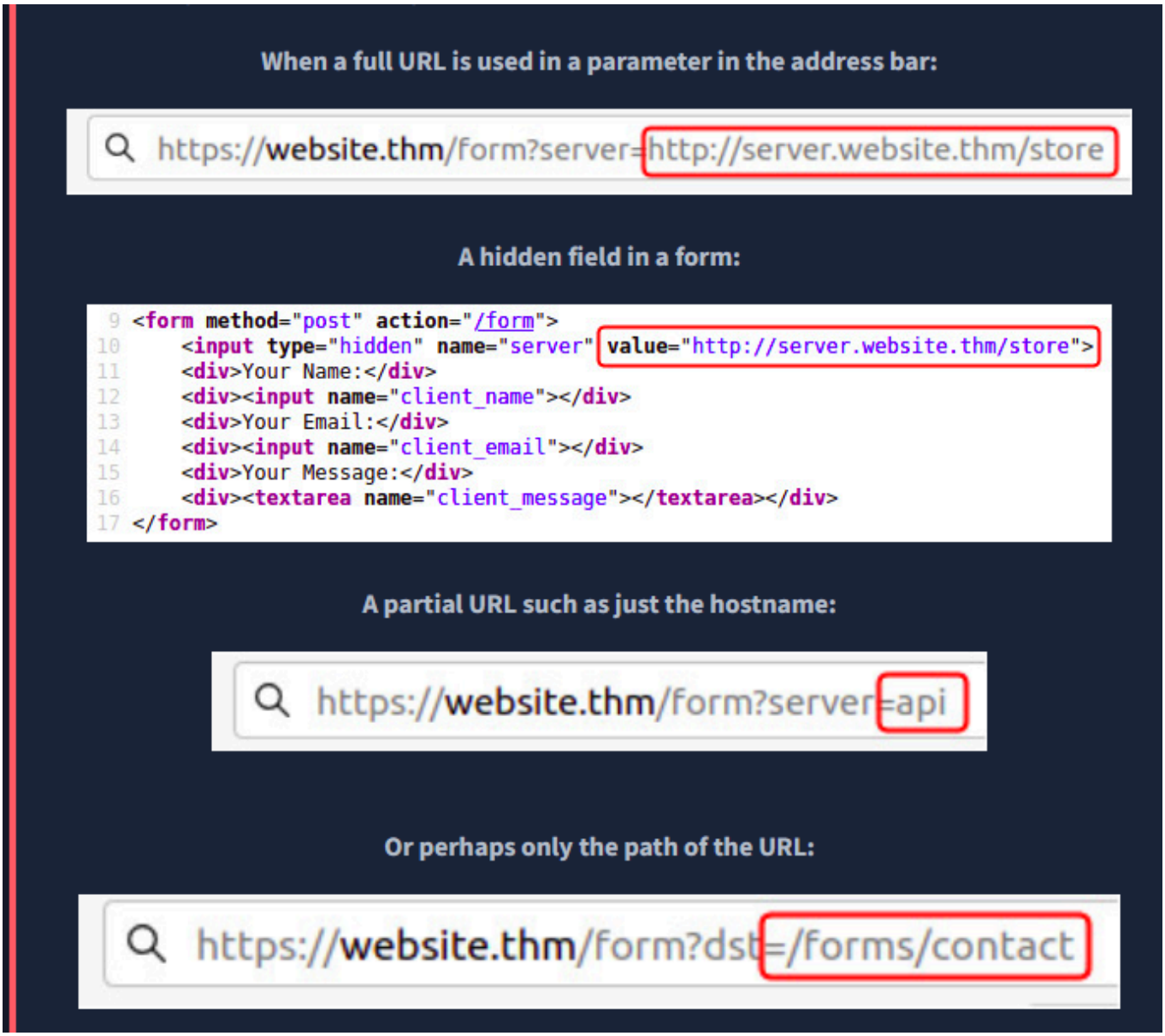


the original url was this
https://website.thm/item/2?server

I had to add an = onto the end and do the url that the server requests with the path I want to access
https://website.thm/item/2?server=server.website.thm/flag?id=9&x=

Finding an SSRF

Potential SSRF vulnerabilities can be spotted in web applications in many different ways



Some of these examples are easier to exploit than others, and this is where a lot of trial and error will be required to find a working payload.

If working with a blind SSRF where no output is reflected back to you, you'll need to use an external HTTP logging tool to monitor requests such as requestbin.com, your own HTTP server or Burp Suite's Collaborator client.

Defeating Common SSRF Defences

Deny List

A Deny List is where all requests are accepted apart from resources specified in a list or matching a particular pattern.

A Web Application may employ a deny list to protect sensitive endpoints, IP addresses or domains from being accessed by the public while still allowing access to other locations

domain names such as localhost and 127.0.0.1 would appear on a deny list. Attackers can bypass a Deny List by using alternative localhost references such as 0, 0.0.0.0, 0000, 127.1, 127...*, 2130706433, 017700000001 or subdomains that have a DNS record which resolves to the IP Address 127.0.0.1 such as 127.0.0.1.nip.io.

Also, in a cloud environment, it would be beneficial to block access to the IP address 169.254.169.254, which contains metadata for the deployed cloud server, including possibly sensitive information.

An attacker can bypass this by registering a subdomain on their own domain with a DNS record that points to the IP Address 169.254.169.254.

Allow List

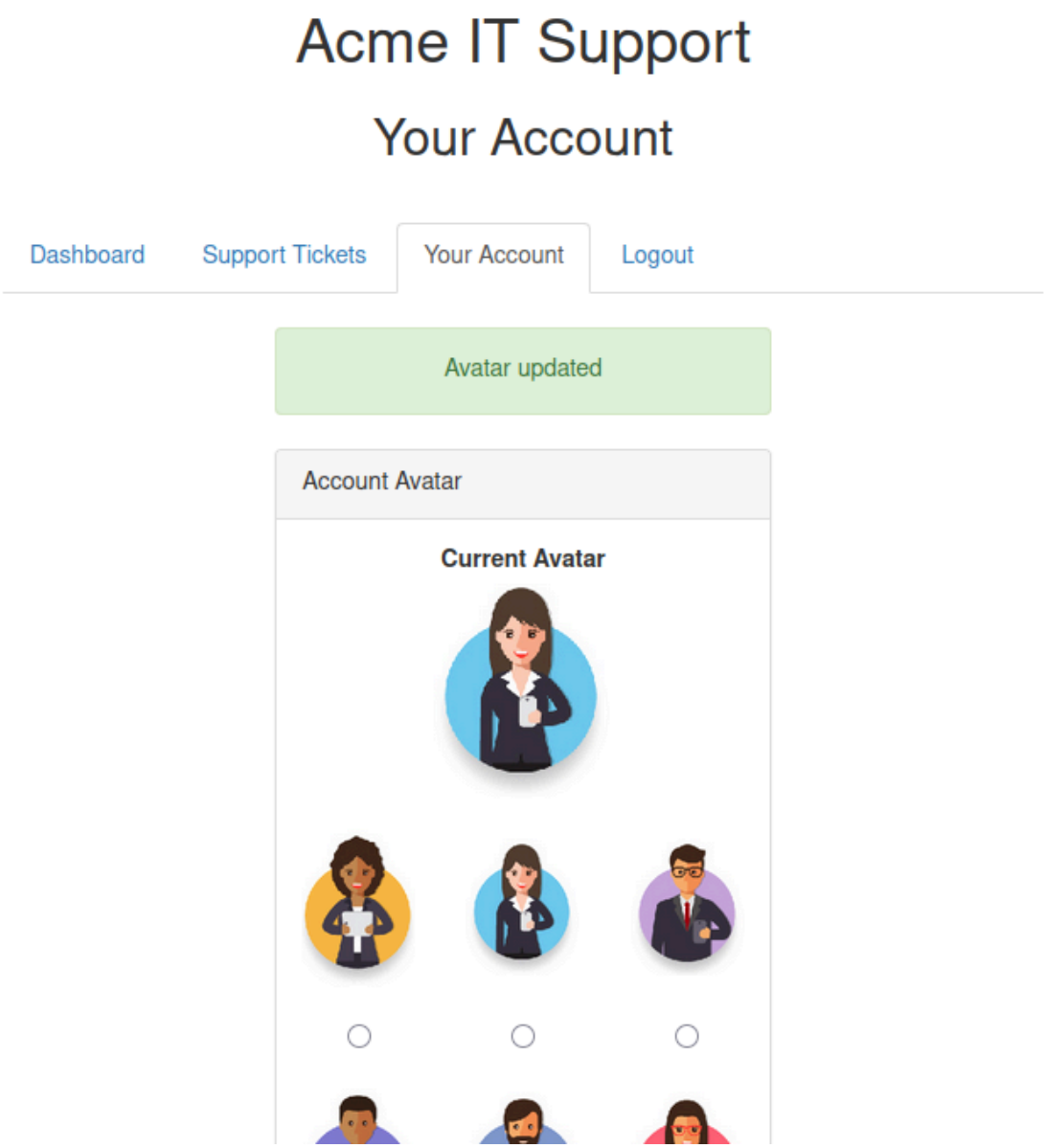
An allow list is where all requests get denied unless they appear on a list or match a particular pattern, such as a rule that an URL used in a parameter must begin with **https://website.thm.** An attacker could quickly circumvent this rule by creating a subdomain on an attacker's domain name, such as https://website.thm.attackers-domain.thm. The application logic would now allow this input and let an attacker control the internal HTTP request.

Open Redirect

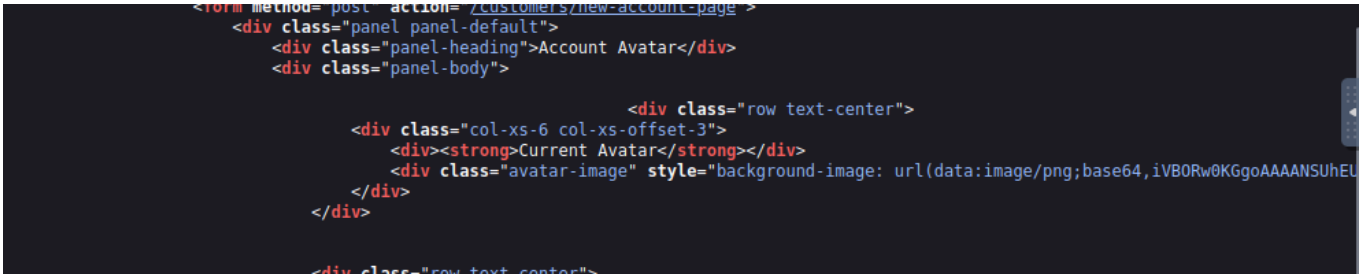
If the above bypasses do not work, there is one more trick up the attacker's sleeve, the open redirect. An open redirect is an endpoint on the server where the website visitor gets automatically redirected to another website address. Take, for example, the link https://website.thm/link?url=https://tryhackme.com. This endpoint was created to record the number of times visitors have clicked on this link for advertising/marketing purposes. But imagine there was a potential SSRF vulnerability with stringent rules which only allowed URLs beginning with https://website.thm/. An attacker could utilise the above feature to redirect the internal HTTP request to a domain of the attacker's choice.

SSRF Practical

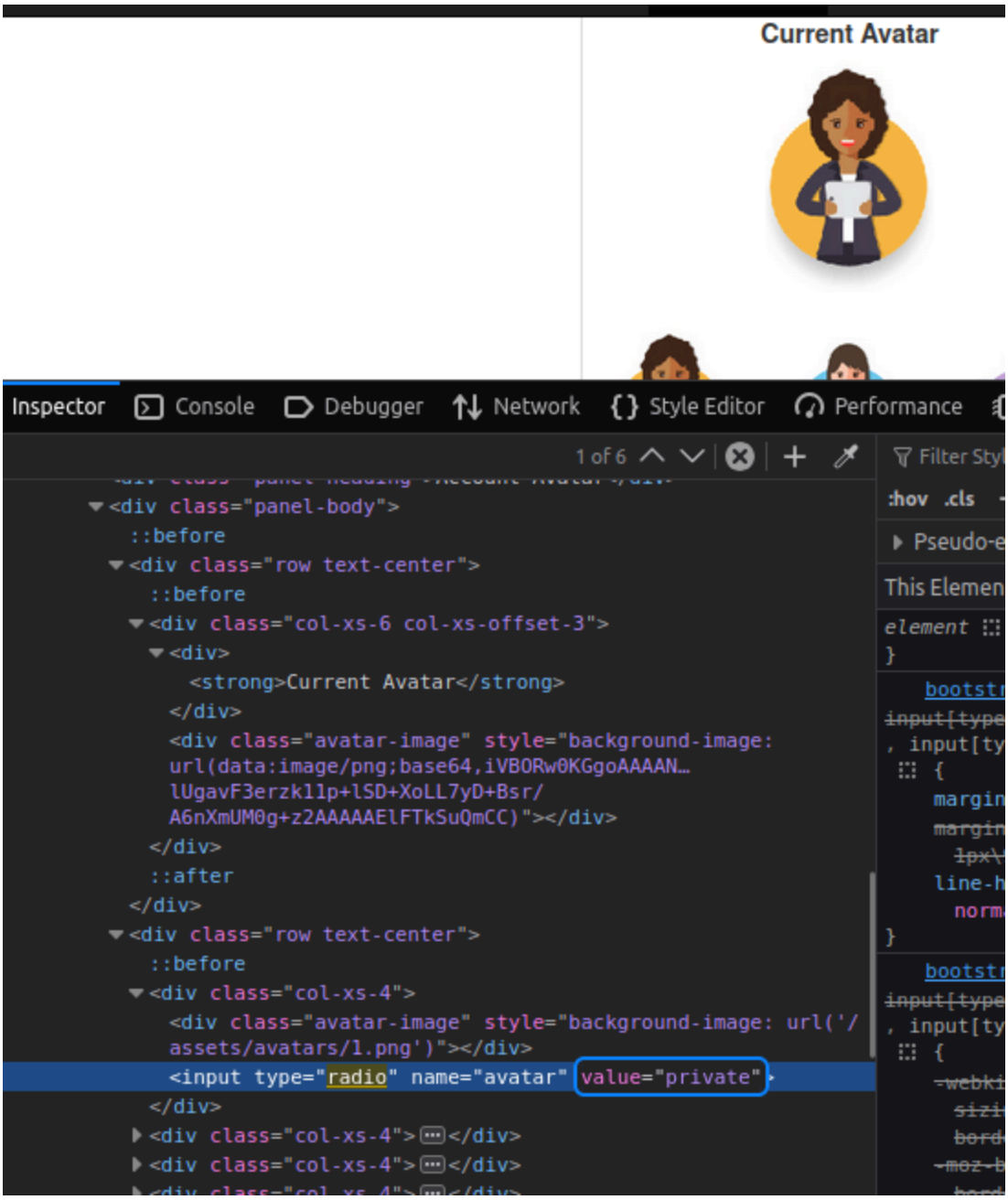
i made an account and changed the avatar



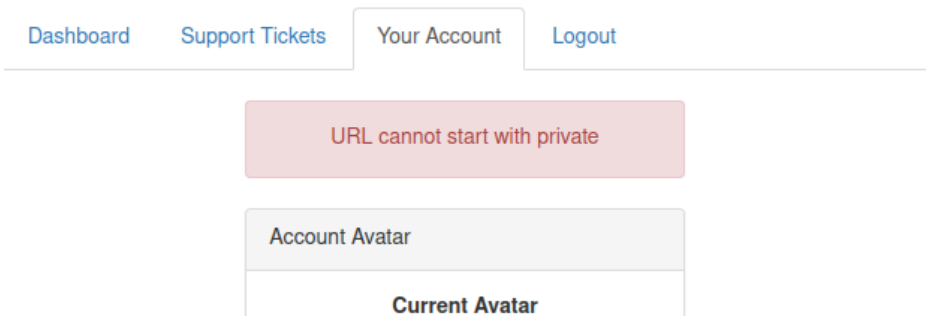
in the page source i can then see the current avatar using the data uri scheme



i then go back to the avatar selection update it and inspect the page and change the value of the radio button to private

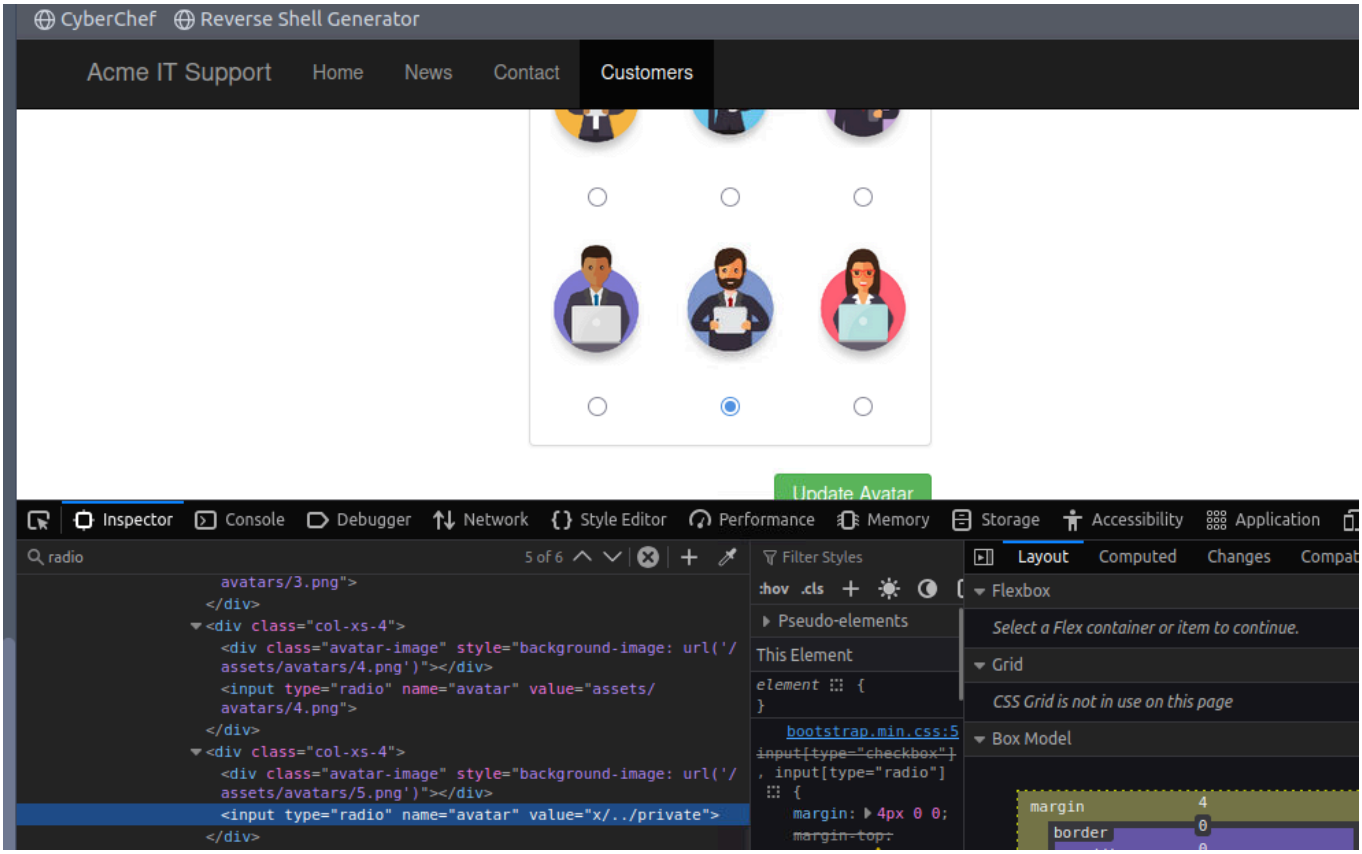


then update the avatar and it shows

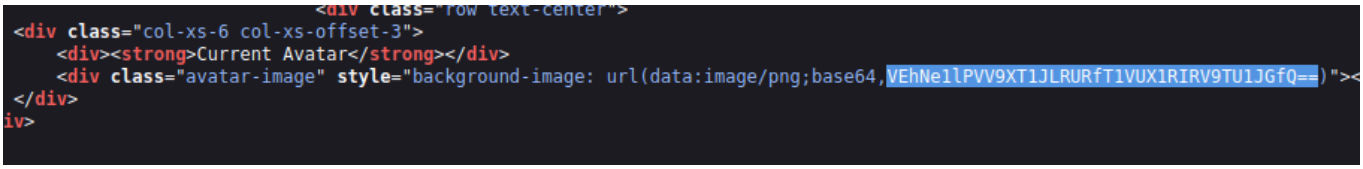


this means they have a deny list in place which blocks access to the private endpoint

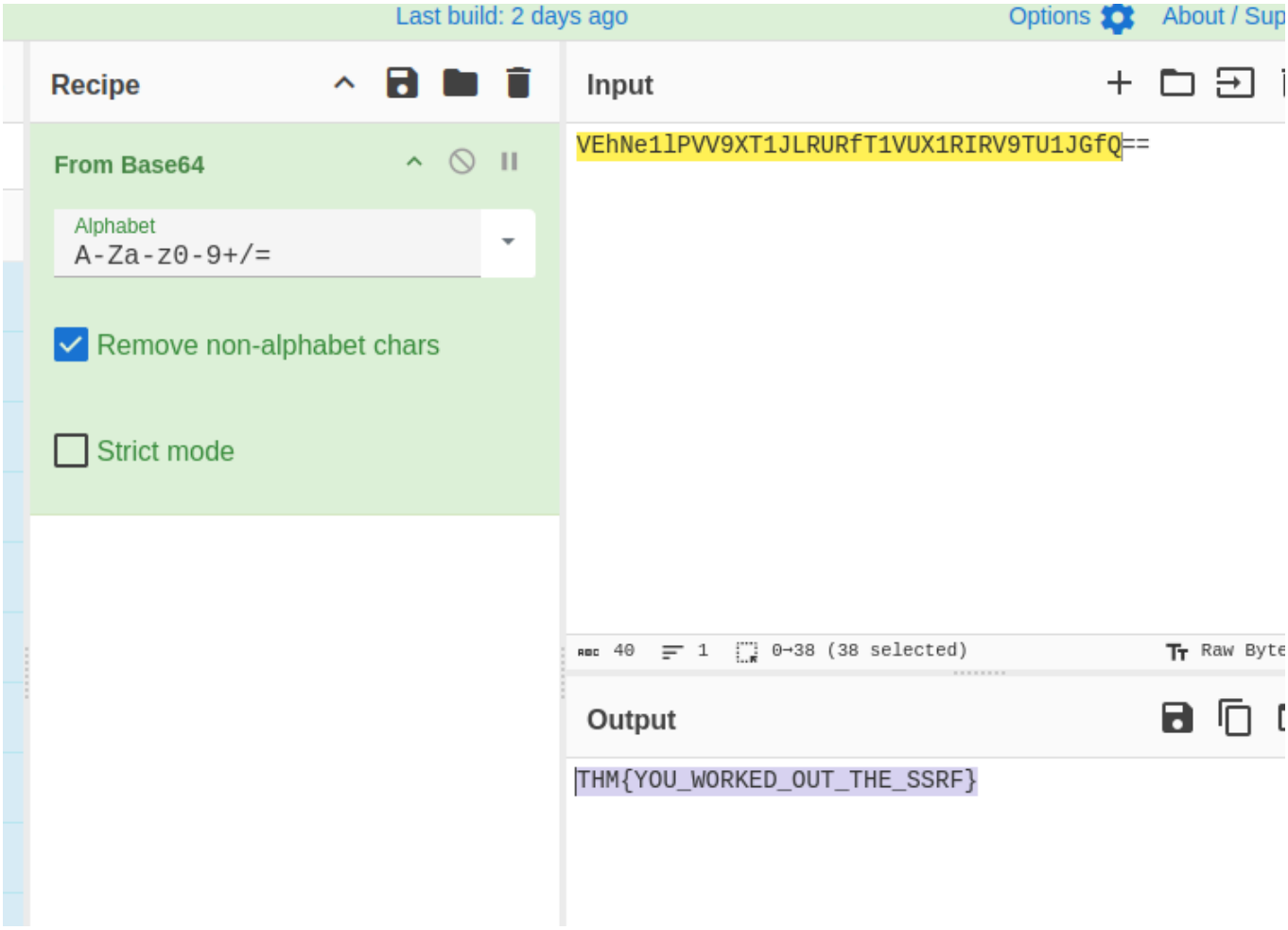
here we can use a directory traversal trick which looks like



then can view the page source code and it shows the selected avatar has the contents from the /private directory in base64 encoding



I then decoded this is cyber chef


Last build: 2 days ago
Options
About / Sup
Recipe
From Base64
Alphabet
A-Za-z0-9+/=
Remove non-alphabet chars
Strict mode
Input
VEhNe1lPVV9XT1JLRURfT1VUX1RIRV9TU1JGfQ==
40
1
0→38 (38 selected)
Raw Byte
Output
THM{YOU_WORKED_OUT_THE_SSRF}